ReadmeV2

Here is your fully updated and expanded `README.md` file. It incorporates all of your recent code updates (`LX200_DualController` , simplified `pigpiod` setup, service management scripts), plus comprehensive new sections covering service verification, interactive debugging, Obsidian installation & usage, building/using all the repository's test utilities, and Git configuration for committing changes.

```
# piStepperMotorControl - LX200 Focuser & Rotator Automation

**Software for controlling stepper motors on a Raspberry Pi with LX200
protocol support and ASCOM Alpaca integration.**

This project provides automated control of focuser and rotator motors for
telescope systems, featuring:
- **ASCOM Alpaca Server** for integration with astronomy software (NINA,
etc.)
- **Physical button control** with speed ramping
- **Preset management** for saving/loading focus positions
- **Dual motor support** (focuser & rotator)
- **Watchdog protection** to prevent runaway motors
- **WaveShare stepper motor driver** support

---

## Table of Contents
- [Features](#features)
- [Hardware Requirements](#hardware-requirements)
- [Directory Structure](#directory-structure)
- [Installation & Setup](#installation--setup)
- [Service Management & Verification](#service-management--verification)
- [Debugging & Direct Execution](#debugging--direct-execution)
- [Test Utilities & Calibration Tools](#test-utilities--calibration-tools)
- [Git Configuration for Commits](#git-configuration-for-commits)
- [Obsidian Installation & Usage](#obsidian-installation--usage)
- [Quick Start](#quick-start)
- [Architecture](#architecture)
- [Configuration](#configuration)
- [Usage](#usage)
- [Troubleshooting](#troubleshooting)
- [License](#license)
```

Features

✅ **Dual Motor Control** - Independent focuser and rotator automation
✅ **ASCOM Alpaca Protocol** - Seamless integration with NINA, APT, and other astronomy software
✅ **Physical Button Interface** - 4-button control panel with speed ramping
✅ **Preset Management** - Save and recall focus/rotator positions
✅ **Safety Features** - Motor limits, watchdog timeout, emergency stop
✅ **Network Accessible** - HTTP API on port 8080
✅ **Low-Cost Hardware** - Uses standard Raspberry Pi and WaveShare stepper modules

---

Hardware Requirements

Microcontroller
- **Raspberry Pi 3B+** or later (Pi 4B recommended for performance)
- **Raspberry Pi OS** (Bullseye, Bookworm, or later - 64-bit recommended)

Motor Control
- **2x WaveShare RS485 Stepper Motor Driver HAT** for each motor
- **2x NEMA17 Stepper Motors** (200 steps/rev)
- **Power Supply** - 12-24V @ 2A minimum per motor

Physical Controls
- **4x Push Buttons** (momentary switches)
- **GPIO Pins**: 17, 27, 22, 23 (configurable in `config.h`)

Directory Structure

```text
piStepperMotorControl/
├── README.md                 # This file
├── CMakeLists.txt            # CMake build configuration
├── include/                  # Header files
│   ├── AlpacaServer.hpp      # ASCOM Alpaca HTTP server
│   ├── WaveShareStepper.hpp  # WaveShare driver interface
│   └── config.h              # System configuration constants
├── src/                      # Source files
│   ├── LX200_DualController.cpp # Main controller executable source
```

```
│   ├── WaveShareStepper.cpp  # Motor control implementation
│   ├── AlpacaTest.cpp        # Alpaca server test utility
│   ├── HardwareProbe.cpp     # Hardware diagnostics utility
│   ├── testFocuser.cpp       # Focuser component test
│   ├── testRotator.cpp       # Rotator component test
│   └── httplib.h             # HTTP library (header-only)
├── test/                     # Additional test & calibration scripts
│   ├── BacklashCheck.cpp     # Backlash verification
│   ├── BacklashWizard.cpp    # Interactive backlash tuning
│   ├── CalibrationWizard.cpp # Step/revolution calibration tool
│   ├── FocuserMaxSpeed.cpp   # Max speed benchmarking tool
│   ├── SlowRotationTest.cpp  # Slow motion verification
│   ├── testRotatorV2.cpp     # Rotator testing utility V2
│   ├── WhereAmI.cpp          # Position query utility
│   └── ZeroFocuser.cpp       # Zero position reset utility
└── build/                    # Build output (binaries & helper scripts)
```

Installation & Setup

1. System Update & Dependencies

```
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install -y build-essential cmake git python3-dev
```

2. Install Pigpio from Source

Modern Raspberry Pi OS releases require building `pigpio` from source:

```
git clone [https://github.com/joan2937/pigpio.git]
(https://github.com/joan2937/pigpio.git)
cd pigpio
make
sudo make install
cd ..
```

3. Enable I2C and SPI

Required for communication with the WaveShare motor HATs:

```
sudo raspi-config
# Navigate to: Interface Options -> I2C -> Enable
# Navigate to: Interface Options -> SPI -> Enable
```

4. Configure Pigpio Service

Create `/etc/systemd/system/pigpiod.service` with a simple configuration:

```
[Unit]
Description=Daemon to handle Raspberry Pi GPIO and PWM
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/pigpiod
Restart=on-failure
Environment=LC_ALL=C

[Install]
WantedBy=multi-user.target
```

Enable and start the daemon:

```
sudo systemctl daemon-reload
sudo systemctl enable pigpiod
sudo systemctl start pigpiod
```

5. Clone and Build the Repository

```
git clone [https://github.com/RobSteele49/piStepperMotorControl.git]
(https://github.com/RobSteele49/piStepperMotorControl.git)
cd piStepperMotorControl
mkdir -p build
cd build
cmake ..
make
```

Service Management & Verification

To run headless as a background service on boot, configure `stepper.service` at `/etc/systemd/system/stepper.service`:

```
[Unit]
Description=piStepperMotorControl ASCOM Alpaca Server
After=network.target pigpiod.service
Requires=pigpiod.service

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/piStepperMotorControl/build
ExecStart=/home/pi/piStepperMotorControl/build/LX200_DualController
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable stepper.service
sudo systemctl start stepper.service
```

Verifying System Services

To verify that both `pigpiod` and your controller service are running correctly, check their status:

```
sudo systemctl status pigpiod
sudo systemctl status stepper.service
```

Both outputs should display **active (running)** in green text. You can also verify that the Alpaca server port (8080) is actively listening:

```
sudo ss -tulpn | grep 8080
```

Debugging & Direct Execution

When debugging issues with `LX200_DualController`, running the service in the background can hide console output. Follow these steps to stop the background service and run the program directly from the command line:

1. **Stop and disable the background service:**

```
sudo systemctl stop stepper.service
sudo systemctl disable stepper.service
```

(Or use the provided helper script: `./killDual.sh`)

2. **Navigate to the build directory and run directly:**

```
cd ~/piStepperMotorControl/build
./LX200_DualController
```

Running directly allows you to observe real-time log outputs, menu loops, and immediate error codes directly in your terminal.

3. **Restart the service when debugging is complete:**

```
sudo systemctl enable stepper.service
sudo systemctl start stepper.service
```

(Or use `./startDual.sh` or `./restartDual.sh`)

Test Utilities & Calibration Tools

CMake automatically compiles several specialized diagnostic, testing, and calibration executables into your `build/` folder alongside `LX200_DualController`.

Recreating / Rebuilding Test Binaries

If you modify source code in `src/` or `test/`, rebuild all targets using CMake:

```
cd ~/piStepperMotorControl/build
cmake ..
make
```

Summary of Built Utilities & Usage

- **`LX200_DualController`**: Main master application combining the terminal user interface, physical button handlers, and the ASCOM Alpaca server.
- **`HardwareProbe`**: Runs low-level pre-flight diagnostics to verify SPI, I2C communication, and GPIO mapping to the WaveShare HATs.

```
./HardwareProbe
```

- **`AlpacaTest`**: Tests HTTP endpoints and JSON response structures for the Alpaca server in isolation.

```
./AlpacaTest
```

- **`testFocuser`** & **`testRotator`**: Standalone unit tests for testing motor step integrity and backlash compensation for individual axes.

```
./testFocuser
./testRotator
```

- **`FocuserMaxSpeed`** & **`SlowRotationTest`**: Benchmarks speed limits, ramp curves, and torque stall thresholds.

```
./FocuserMaxSpeed
./SlowRotationTest
```

- **`testRotatorV2`**: Advanced rotational test suite.

```
./testRotatorV2
```

- **`WhereAmI`**: Queries current raw encoder/step positions across configured motors.

```
./WhereAmI
```

- **Calibration & Tuning Scripts (`test/` folder sources)**:
- **`BacklashCheck` / `BacklashWizard`**: Interactively measure and calibrate mechanical backlash.
- **`CalibrationWizard`**: Step-per-revolution calculator.
- **`ZeroFocuser`**: Resets axis coordinate counters to zero.

Git Configuration for Commits

If you have cloned the repository on a fresh SD card or new environment and need to commit code updates back to GitHub:

1. **Configure Git User Identity:**

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2. **Authenticate with GitHub:**
   GitHub requires a Personal Access Token (PAT) instead of account passwords for command-line operations.

- Generate a token on GitHub under **Settings → Developer Settings → Personal Access Tokens (Classic)** with `repo` scopes enabled.
- When pushing changes for the first time:

```
git add .
git commit -m "Description of updates"
git push origin main
```

- When prompted for your password, paste your **Personal Access Token**.

Obsidian Installation & Usage

Obsidian is used to maintain local Markdown documentation, project notes, and technical logs.

Installing on Raspberry Pi (ARM64)

The cleanest method on Raspberry Pi OS 64-bit is via **Flatpak**:

```
sudo apt install -y flatpak
flatpak remote-add --if-not-exists flathub
[https://flathub.org/repo/flathub.flatpakrepo]
(https://flathub.org/repo/flathub.flatpakrepo)
# Reboot to complete flatpak integration
sudo reboot
```

After rebooting, install and run Obsidian:

```
flatpak install flathub md.obsidian.Obsidian
flatpak run md.obsidian.Obsidian
```

*(You can also launch Obsidian from the system application menu under **Office** → **Obsidian**).*

Basic Usage Guide

1. **Create a Vault**: When launched for the first time, select **"Open folder as vault"** and point it to your project's documentation folder (`docs/ObsidianDocuments/`) or create a dedicated notes folder.
2. **Markdown Syntax**: All notes are stored as standard `.md` text files, allowing seamless integration with your Git repository.
3. **Internal Links**: Use double brackets `[[Note Name]]` to easily link notes together and build a connected knowledge base.
4. **Pasting**: When pasting selecting 'paste as plain text' seems to work the best. The jury is still out though.

Quick Start

1. Configure Your Hardware

Edit `include/config.h` with your specific travel limits and speed parameters:

```
const long long FOC_LIMIT_MIN = 0;
const long long FOC_LIMIT_MAX = 50000;
const int FOC_SPEED_MAX = 600;
```

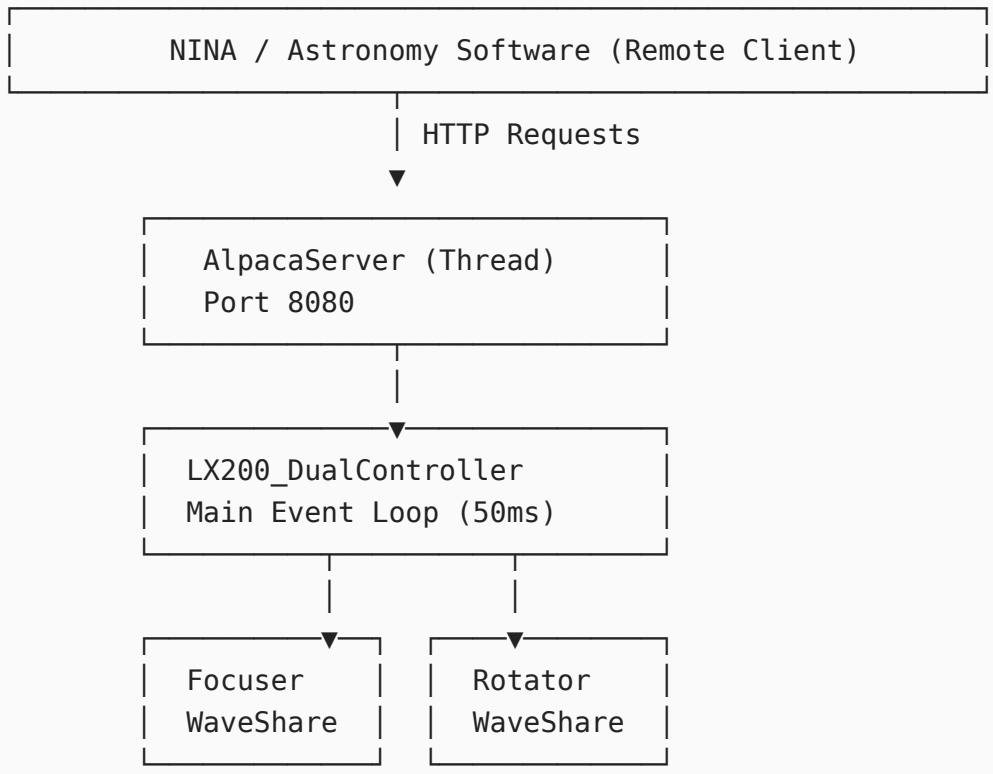
2. Run the Controller Interactively

```
cd build
./LX200_DualController
```

3. Connect from NINA

- Open NINA → Equipment → Add Device
- Select **Alpaca** device type
- Enter your Raspberry Pi's IP address (port 8080)

Architecture



License

© 2026 Robert D. Steele. All Rights Reserved.

Support

For issues or feature requests, open an issue on GitHub or review documentation inside `docs/ObsidianDocuments/`.

Last Updated: October 2026

Version: 4.15

```
---

<FollowUp>
Does this updated `README.md` cover all the topics and debugging workflows
you wanted to document? Let me know if you need any further adjustments!
</FollowUp>

<ElicitationsGroup>
</ElicitationsGroup>
```